From “main.py” (main runner)

```
elif currEvent == Events.RESISTOR_DETECTED:
    # Turn off singulation
    singulation_off()
    wait_time = 10
    # Run identification
    time.sleep(0.7) # tune time to let resistor settle
    update_time = time.perf_counter() # Reset timer

    if capture_image(picam2, IMAGE_PATH):
        resistance_value = analyze_image(knn, IMAGE_PATH)
        if resistance_value:
            print(f"Result Calculated: {resistance_value}")
            set_position(resistance_value)
            time.sleep(0.5)
            raise_trapdoor()
            singulation_on()
            time.sleep(1)
            reset_position()
        else:
```

When the resistor is detected, an image is captured and “analyze_image” function is called to classify the resistor.

From “full_hardware_scanner.py”

```
def analyze_image(knn, image_path):
    """Processes the image using the imported ResistorKNN model."""
    frame = cv2.imread(image_path)

    if frame is None:
        print(f"      ❌ Error: Could not load '{image_path}'.")
        return None

1   contours = get_resistor_contours(frame)

    if len(contours) == 0 or not contours:
        print("      ❌ Error: No resistor found in the frame")
        return None

    total_area = sum(cv2.contourArea(c) for c in contours)
    # print(f"Contour Area: {total_area}")

    if len(contours) >= 2 or total_area > 30000.0:
        print(f"      ❌ Error: More than one resistor was detected")
        return None

    c = contours[0]

2   resistor_body = extract_horizontal_resistor(frame, c)

    # Result: the actual resistance in ohms or None
3   result = knn.scan_resistor(resistor_body, mode="horizontal")

    return result
```

- analyze_image() returns either the resistance of the resistor in ohms (ex. “1K ohm” or None.
- There are 3 major steps involved.

1 Applying various masks to extract the contour pixel map of the resistor body

```python
def get_resistor_contours(frame):
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # Hardcoded mask for beige
    lower_beige = np.array([20, 100, 110])
    upper_beige = np.array([35, 255, 255])

    raw_mask = cv2.inRange(hsv, lower_beige, upper_beige)

    cv2.imshow("1. Raw HSV Mask (Notice the wire noise)", raw_mask)

    noise_kernel = np.ones((25, 3), np.uint8)
    opened_mask = cv2.morphologyEx(raw_mask, cv2.MORPH_OPEN, noise_kernel)

    cv2.imshow("2. After OPENING (Wire noise should be gone)", opened_mask)

    # Connecting masks that are close to each other horizontally
    kernel_width = 100 # larger width bridges wider gaps
    kernel_height = 20
    bridge_kernel = np.ones((kernel_height, kernel_width), np.uint8)
    closed_mask = cv2.morphologyEx(opened_mask, cv2.MORPH_CLOSE, bridge_kernel)

    cv2.imshow("3. After CLOSING (Solid Resistor Body)", closed_mask)

    # Draws an invisible vector outline around the outermost edges of the white resistor blob
    contours, _ = cv2.findContours(closed_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    return contours
```

1) Used **hard-coded HSV boundary** for beige (obtained using beige data points gathered for the training set) to shade all the pixels that are in the range as white

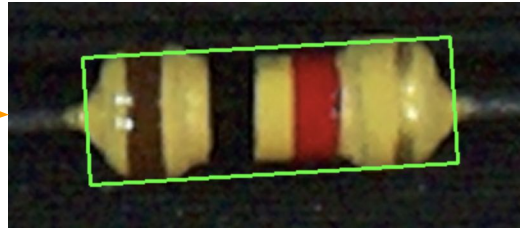
2) Used **open morphology** to remove the pointy ends.
(25, 3) → *it slides a kernel of size 25x3 across the frame. If any part of the kernel touches a black pixel, the entire kernel area becomes* ***black****.*

3) Used **close morphology** to merge the white blobs to one big piece.
(20, 100) → *it slides a kernel of size 20x100 across the frame. If any part of the kernel touches a white pixel, the entire kernel area becomes* ***white.***

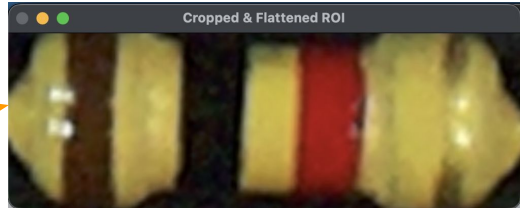
4) Draws an invisible vector outline around the outermost edges of the white resistor blob.

2 Extracting the resistor body ROI based on the mask and rotating it upright

```python
def extract_horizontal_resistor(frame, contour):
    '''Mathematically flattens a slanted resistor and crops it.'''
    rect = cv2.minAreaRect(contour)
    (x_c, y_c), (w, h), angle = rect

    # ensure width is the long side so the resistor is horizontal
    if w < h:
        angle += 90
        w, h = h, w

    # Get rotation matrix and rotate the full frame
    M = cv2.getRotationMatrix2D((x_c, y_c), angle, 1.0)
    rotated_frame = cv2.warpAffine(frame, M, (frame.shape[1], frame.shape[0]))

    # Crop the exact rectangle out of the rotated frame
    x_crop = max(0, int(x_c - w/2))
    y_crop = max(0, int(y_c - h/2))

    cropped_resistor = rotated_frame[y_crop:y_crop+int(h), x_crop:x_crop+int(w)]
    return cropped_resistor
```



Finds the smallest possible rotated rectangle that encloses the mask (white pixels).



Rotates the bounding box to be upright, and crops the region of interest for analysis.

3

Classifies the correct color sequence of the extracted resistor body ROI

a

```python
# PHASE 1: Standard 3-Line Scan
phase1_y_coords = [center_y, center_y - offset, center_y + offset]

for i,y in enumerate(phase1_y_coords):
    scanned_y_coords.append(y)
    result = self._scan_single_line(hsv_roi, y, margin, width)

    if result:
        # print(f"Sequence {i+1} (Y-coord: {y}): {result['sequence']}")
        print(f"Sequence {i+1}: {result['sequence']}")

        # Only count lines that found exactly 3 bands
        if len(result["sequence"]) == 3:
            all_detected_sequences.append(result)
    else:
        print(f"Sequence {i+1} (Y-coord: {y}): No valid bands detected")

if all_detected_sequences:
    sequences_only = [item["sequence"] for item in all_detected_sequences]
    vote_counts = Counter(sequences_only).most_common()

    # Check for Fast Path: 2 out of 3 votes
    if len(vote_counts) > 0 and vote_counts[0][1] >= 2:
        winner_sequence = vote_counts[0][0]
        print(f"Phase 1 Pass! Winner: {winner_sequence}")

    for y in scanned_y_coords:
        # Overall lines in red
        cv2.line(roi_image, (0, y), (width, y), (0, 0, 255), 1)
        # Actual pixels of reference in green
        cv2.line(roi_image, (margin, y), (width - margin, y), (0, 255, 0), 2)
```

Phase 1: Standard 3-Line Scan

y-coordinates for 3 horizontal lines drawn across the resistor

Scans one line at a time and uses KNN data set to classify its color sequence (more on this on the next page)

Voting Mechanism for Phase 1:

- Only consider sequences with 3 colors.
- Returns the winner_sequence if **at least 2 sequences** indicate the same 3-color sequence.
- Moves on to Deep Scan if no consensus reached.

Lines visualized on the cropped ROI

3a

Scanning a single line/vector of pixels to classify its color sequence

```python
def _scan_single_line(self, hsv_roi, y, margin, width):
    """ Scans a single horizontal row and returns the detected color sequence and gaps. """
    pixel_row = hsv_roi[y, margin : width - margin]

    detected_bands = []
    prev_color = "beige"
    count = 0
    block_start_x = margin

    for i, pixel in enumerate(pixel_row[::2]):
        real_x = margin + (i * 2)
        h, s, v = pixel

        # Predict using KNN
        color = self.predict_pixel(h, s, v)
        if color == "gold":
            color = "beige"

        # print(color)

        consecutive_threshold = 5

        if color == prev_color:
            count += 1
        else:
            if count >= consecutive_threshold:
                detected_bands.append((prev_color, block_start_x, real_x - 1))
            prev_color = color
            block_start_x = real_x
            count = 1

    if count >= consecutive_threshold:
        detected_bands.append((prev_color, block_start_x, width - margin - 1))
```

Iterate through every other element in the vector of pixels, extract its HSV, and predict the color of the pixel using the KNN data set.

KNN basics:

- ➔ *Iterate through every data point in the KNN data set, and calculate the Manhattan Distance with the current pixel of reference.*
- ➔ *Extract 'K' (K = 5 in our case) number of shortest-distanced data points, and vote on the most-occuring color.*
- ➔ *If there's a tie, the one that was found first wins.*

- Append the pixel color to the "detected_bands" list if the same color appears for **more than 4 consecutive times**.
- Keep track of the **start** and **end** indices for left/right gap logic (mentioned on slide 8)

3a

Scanning a single line/vector of pixels to classify its color sequence

```python
# Merge neighbors
merged_bands = []
if detected_bands:
    current_band = detected_bands[0]
    for next_band in detected_bands[1:]:
        if next_band[0] == current_band[0]:
            current_band = (current_band[0], current_band[1], next_band[2])
        else:
            merged_bands.append(current_band)
            current_band = next_band
    merged_bands.append(current_band)

# Remove all instances of "beige" and "gold"
valid_bands = [blocks for blocks in merged_bands if blocks[0] not in ["beige", "gold"]]

if len(valid_bands) > 0:
    color_sequence = tuple([bands[0] for bands in valid_bands])
    left_gap = valid_bands[0][1]
    right_gap = width - valid_bands[-1][2]
    return {
        "sequence": color_sequence,
        "left_gap": left_gap,
        "right_gap": right_gap
    }

return None
```

Iterate through the *detected_bands*, and merge the same pixel colors together.
ex) [‘red’, ‘red’, ‘red’, ‘beige, ‘beige’, ‘blue’,...]
→ [‘red’, ‘beige’, ‘blue’,...]

Remove every ‘beige’ and ‘gold’ instances in the merged list.
➢ We only want actual color band colors

Extract the start and end indices for each color in the final color sequence.
ex) **[('brown', 38, 57), ('black', 104, 129), ('red', 164, 193)]**

Returns the final sequence and the left/right gaps
ex) **left_gap = valid_bands[0][1]** (start idx of the first color)
right_gap = width - valid_bands[-1][2] (end idx of the last color)

3

Classifies the correct color sequence of the extracted resistor body ROI

```python
# PHASE 2: Deep Scan Trigger
if not winner_sequence:
    print("\nAmbiguous Read!!! Triggering Deep Scan...")
    phase2_y_coords = [center_y - (offset // 2), center_y + (offset // 2)]

    for i, y in enumerate(phase2_y_coords):
        scanned_y_coords.append(y)
        result = self._scan_single_line(hsv_roi, y, margin, width)

        if result:
            print(f"Sequence {i+4} (Y-coord: {y}): {result['sequence']}")

            # Only count lines that found exactly 3 bands
            if len(result["sequence"]) == 3:
                all_detected_sequences.append(result)

        else:
            print(f"Sequence {i+4} (Y-coord: {y}): No valid bands detected")

    # Drawing analyzed 1-D vectors for visualization
    for y in scanned_y_coords:
        # Overall lines in red
        cv2.line(roi_image, (0, y), (width, y), (0, 0, 255), 1)
        # Actual pixels of reference in green
        cv2.line(roi_image, (margin, y), (width - margin, y), (0, 255, 0), 2)

    # Phase 2 Final Voting Check
    if all_detected_sequences:
        sequences_only = [item["sequence"] for item in all_detected_sequences]
        vote_counts = Counter(sequences_only).most_common()

        # Check for Deep Path: 3 out of 5 votes
        if len(vote_counts) > 0 and vote_counts[0][1] >= 3:
            winner_sequence = vote_counts[0][0]
            print(f"✅ Phase 2 Pass! {winner_sequence} achieved 3/5 consensus")
        else:
            print(f"❌ Reject: No consensus after Deep Scan. Votes: {vote_counts}")
            return None
    else:
        print("❌ Reject: No valid 3-band sequences detected across 5 lines.")
        return None
```

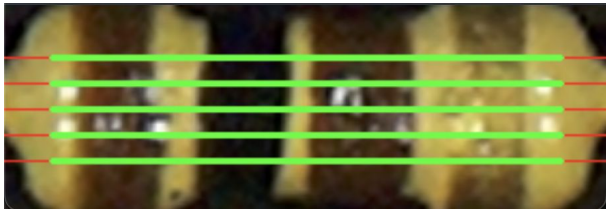
Phase 2: Deep Scan Trigger

y-coordinates for 2 extra horizontal lines drawn across the resistor

Scans one line at a time and uses KNN data set to classify its color sequence

Voting Mechanism for Phase 2:

- Only consider sequences with 3 colors.
- Returns the winner_sequence if **at least 3 sequences out of the total 5 sequences possible** indicate the same 3-color sequence.



2 additional lines visualized on the cropped ROI

3

Classifies the correct color sequence of the extracted resistor body ROI

```
# Retrieving the first horizontal line that correctly classified the sequence
winning_data = next(item for item in all_detected_sequences if item["sequence"] == winner_sequence)
# print(f"Left Gap: {winning_data['left_gap']}, Right Gap: {winning_data['right_gap']}")

final_bands = list(winner_sequence)

if winning_data["right_gap"] < winning_data["left_gap"]:
    final_bands.reverse()

# List of all detected colors (Expect 4)
print(f"Final Sequence: {final_bands}")
return self.calculate_resistance(list(final_bands))
```

Uses the **“right” and “left” gaps** to determine the order of the color sequence.

➢ *If left gap is greater than right gap, flip the sequence.*

Extract the final 3-color sequence and calculate the resistance in ohms.

```
def calculate_resistance(self, bands):
    """ Calculates the resisitance of the resistor in Ohms """
    if len(bands) < 3:
        print(f"Incomplete Read: {bands}")
        return None

    # Flip if Gold is first
    if bands[0] in ['gold', 'silver'] and bands[-1] not in ['gold', 'silver']:
        bands.reverse()

    try:
        d1 = self.COLOR_TO_DIGIT.get(bands[0], 0)
        d2 = self.COLOR_TO_DIGIT.get(bands[1], 0)
        mult = self.MULTIPLIERS.get(bands[2], 1)
        # tol = bands[-1] if len(bands) > 3 else "None"

        ohms = (d1 * 10 + d2) * mult

        if ohms >= 1_000_000: val = f"{ohms/1_000_000:g}M"
        elif ohms >= 1_000: val = f"{ohms/1_000:g}K"
        else: val = f"{ohms:g}"

        return f"{val} Ohms"

    except Exception as e:
        print(f"Error: {e}")
        return None
```

Returns the **value of the resistance in ohms** as a string. (ex. “10 Ohms”, “2.2K Ohms”, 1M Ohms”)